

Back-end em python



Prof^a. Ma. Ana Clara

SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO



Aula 20 - Testes de Unidade



Objetivos da Aula

- Consolidar o uso de testes automatizados
- Criar testes de unidade para métodos orientados a objetos
- Iniciar o desenvolvimento do projeto final com POO e testes
- Estimular organização, clareza e boas práticas em Python



Aula 20 - Testes de Unidade

O que são testes de unidade?

- Testam partes específicas do código isoladamente
- Validação automática de métodos e funções
- Essenciais para garantir qualidade e evitar regressões





Aula 20 - Testes de Unidade

Por que testar?

- Detecção precoce de erros
- Refatoração segura
- Documentação viva do sistema
- Agilidade no desenvolvimento em equipe





Aula 20 - Testes de Unidade



Biblioteca unittest (Python)

```
import unittest

class TesteSimples(unittest.TestCase):
    def test_soma(self):
        self.assertEqual(2 + 3, 5)

if __name__ == '__main__':
    unittest.main()
```



Aula 20 - Testes de Unidade



Métodos de teste mais comuns

- `assertEqual(a, b)`
- `assertTrue(x) / assertFalse(x)`
- `assertRaises(Exception)`
- `assertIn(x, lista) / isinstance(x, tipo)`



Aula 20 - Testes de Unidade



Exemplo práticos - Classe Produto

```
class Produto:
    def __init__(self, nome, preco):
        if preco < 0:
            raise ValueError("Preço inválido")
        self.nome = nome
        self.preco = preco
```



Aula 20 - Testes de Unidade



Testando Produto

```
class TesteProduto(unittest.TestCase):  
    def test_preco_valido(self):  
        p = Produto("Livro", 50)  
        self.assertEqual(p.preco, 50)  
  
    def test_preco_invalido(self):  
        with self.assertRaises(ValueError):  
            Produto("Tênis", -20)
```




Aula 20 - Testes de Unidade



Organização do projeto com testes

```
projeto_final/  
├── produto.py  
├── test_produto.py  
├── usuario.py  
└── test_usuario.py
```

- Código e testes separados
- Executar com: `python -m unittest`



Aula 20 - Testes de Unidade



Organização do projeto com testes

```
$ python -m unittest  
...  
OK
```

- Pode rodar arquivos isolados ou toda a pasta



Aula 20 - Testes de Unidade

Atividade 1 - Criar classe Calculadora + testes

Métodos: `somar`, `subtrair`, `multiplicar`,
`dividir`

Criar arquivo `test_calculadora.py`





Aula 20 - Testes de Unidade



Atividade 2 - Testar exceções personalizadas

Criar validação para nomes vazios ou dados inválidos. Exemplo:

```
if not nome:  
    raise ValueError("Nome obrigatório")
```

Criar teste usando `assertRaises`



Aula 20 - Testes de Unidade



Início do Projeto Final com POO

Parceria com:

O que deve conter:

- 2+ classes
- métodos com regras de negócio
- testes automatizados



Aula 20 - Testes de Unidade



Escopo mínimo obrigatório

- . Orientação a objetos real
- . Testes unitários com cobertura mínima
- . Interação básica via terminal (opcional)
- . Organização de arquivos em módulos



Aula 20 - Testes de Unidade



Planejamento guiado (atividade)

Definir:

- Nome do projeto
- Classes e atributos
- Principais métodos
- Cenários de teste



Aula 20 - Testes de Unidade



Estrutura inicial sugerida

```
projeto/  
├── main.py  
├── classe1.py  
├── classe2.py  
├── test_classe1.py  
└── test_classe2.py
```




Aula 20 - Testes de Unidade



Ciclo de desenvolvimento com testes

1. Criar a funcionalidade
2. Escrever o teste correspondente
3. Rodar os testes
4. Refatorar e corrigir



Aula 20 - Testes de Unidade



Compartilhamento de ideias de projetos

- Apresente a sua ideia
- Já tem ideia de escopo, testes e modularização?



Aula 20 - Testes de Unidade



- Fixamos a importância de testar classes e métodos
- Começamos a construir o projeto final
- Próxima aula: refinamento do projeto e integração entre classes



Aula 20 - Testes de Unidade



Materiais para Estudo

- [Documentação unittest](#)
- YouTube: Curso Dev Aprender – Testes em Python
- Livro: "Test-Driven Development with Python" – Harry Percival
- Artigo prático: <https://realpython.com/python-testing/>



Aula 20 - Testes de Unidade



Alguma dúvida ?

clara.gomes@ufpe.br



SOFTEX
PERNAMBUCO

 **Softex**

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO

GOVERNO FEDERAL
BRASIL
UNIÃO E RECONSTRUÇÃO